Experiment 7: Perform Nested and Complex Queries

Aim: Mastering Subqueries (IN, NOT IN, EXISTS, NOT EXISTS, ANY, ALL), Correlated Subqueries, Derived Tables (FROM clause), HAVING clauses, and Set Operators (UNION, INTERSECT).

1 [Subquery - IN]: Write an SQL query using an IN subquery to find the full names and countries of players who scored more than 50 runs (NRuns > 50) in any single match.

SELECT CONCAT(FName, ' ', LName) AS PlayerName, Country FROM Player WHERE PlayerID IN (SELECT PID FROM Batting WHERE NRuns > 50);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT CONCAT(FName, ' ', LName) AS PlayerName, Country
                                                -> FROM Player
                                                -> WHERE PlayerID IN (
                                                ->     SELECT PID FROM Batting WHERE NRuns > 50
                                                -> );
+-------------------+-----------+
| PlayerName        | Country   |
+-------------------+-----------+
| MS Dhoni          | India     |
| Sanath Jayasuriya | Sri Lanka |
| Sachin Tendulkar  | India     |
+-------------------+-----------+
3 rows in set (0.0049 sec)
```

2 [Subquery - NOT IN]: Retrieve all players (PlayerID, FName, LName, Country) who have NEVER bowled in any match using a NOT IN subquery referencing the Bowling table.

SELECT PlayerID, FName, LName, Country FROM Player WHERE PlayerID NOT IN (SELECT PID FROM Bowling WHERE PID IS NOT NULL);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT PlayerID, FName, LName, Country
                                                -> FROM Player
                                                -> WHERE PlayerID NOT IN (
                                                ->     SELECT PID FROM Bowling WHERE PID IS NOT NULL
                                                -> );
+----------+------------+-----------+--------------+
| PlayerID | FName      | LName     | Country      |
+----------+------------+-----------+--------------+
|    21001 | Virender   | Sehwag    | India        |
|    25001 | MS         | Dhoni     | India        |
|    27001 | Praveen    | Kumar     | India        |
|    27002 | Ishant     | Sharma    | India        |
|    89001 | Sachin     | Tendulkar | India        |
|    90001 | Brain      | Lara      | West Indies  |
|    90002 | Anil       | Kumble    | India        |
|    92001 | Shane      | Warne     | Australia    |
|    92003 | Inzamam    | UI-Huq    | Pakistan     |
|    93002 | Heath      | Streak    | Zimbabwe     |
|    93003 | Gary       | Kirsten   | South Africa |
|    94004 | Stephen    | Fleming   | New Zealand  |
|    95001 | Ricky      | Ponting   | Australia    |
|    95002 | Shaun      | Pollock   | South Africa |
|    95003 | Jacques    | Kallis    | South Africa |
|    96001 | Rahul      | Dravid    | India        |
|    96002 | Herschelle | Gibbs     | South Africa |
|    96003 | Saurav     | Ganguly   | India        |
|    97004 | Daniel     | Vettori   | New Zealand  |
|    98001 | Shahid     | Afridi    | Pakistan     |
|    99002 | Adam       | Gilchrist | Australia    |
|    99003 | Michael    | Vaughan   | England      |
|   300311 | Michael    | Clark     | NULL         |
|   406317 | Sam        | NULL      | India        |
+----------+------------+-----------+--------------+
24 rows in set (0.0049 sec)
```

3 [Subquery - EXISTS]: Write an SQL query using EXISTS to list all matches (MatchID, Ground, Date) where at least one player hit 2 or more sixes (Six >= 2).

SELECT m.MatchID, m.Ground, m.Date FROM ODIMatch m WHERE EXISTS (SELECT 1 FROM Batting b WHERE b.MatchID = m.MatchID AND b.Six >= 2);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT m.MatchID, m.Ground, m.Date
                                                -> FROM ODIMatch m
                                                -> WHERE EXISTS (
                                                ->     SELECT 1 FROM Batting b
                                                ->     WHERE b.MatchID = m.MatchID AND b.Six >= 2
                                                -> );
+---------+----------+------------+
| MatchID | Ground   | Date       |
+---------+----------+------------+
|    2755 | Colombo  | 2008-08-24 |
|    2689 | Brisbane | 2008-03-04 |
+---------+----------+------------+
2 rows in set (0.0017 sec)
```

4 [Subquery - NOT EXISTS]: Retrieve all players (PlayerID, FName, LName) for whom NO bowling record exists in the Bowling table using a correlated NOT EXISTS clause.

SELECT p.PlayerID, p.FName, p.LName FROM Player p WHERE NOT EXISTS (SELECT 1 FROM Bowling b WHERE b.PID = p.PlayerID);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT p.PlayerID, p.FName, p.LName
                                                -> FROM Player p
                                                -> WHERE NOT EXISTS (
                                                ->     SELECT 1 FROM Bowling b
                                                ->     WHERE b.PID = p.PlayerID
                                                -> );
+----------+------------+-----------+
| PlayerID | FName      | LName     |
+----------+------------+-----------+
|    21001 | Virender   | Sehwag    |
|    25001 | MS         | Dhoni     |
|    27001 | Praveen    | Kumar     |
|    27002 | Ishant     | Sharma    |
|    89001 | Sachin     | Tendulkar |
|    90001 | Brain      | Lara      |
|    90002 | Anil       | Kumble    |
|    92001 | Shane      | Warne     |
|    92003 | Inzamam    | UI-Huq    |
|    93002 | Heath      | Streak    |
|    93003 | Gary       | Kirsten   |
|    94004 | Stephen    | Fleming   |
|    95001 | Ricky      | Ponting   |
|    95002 | Shaun      | Pollock   |
|    95003 | Jacques    | Kallis    |
|    96001 | Rahul      | Dravid    |
|    96002 | Herschelle | Gibbs     |
|    96003 | Saurav     | Ganguly   |
|    97004 | Daniel     | Vettori   |
|    98001 | Shahid     | Afridi    |
|    99002 | Adam       | Gilchrist |
|    99003 | Michael    | Vaughan   |
|   300311 | Michael    | Clark     |
|   406317 | Sam        | NULL      |
+----------+------------+-----------+
24 rows in set (0.0018 sec)
```

5 [Scalar Subquery in WHERE]: Find all players (FName, LName, Yborn) who were born before the average birth year (Yborn < AVG(Yborn)) of all registered players.

SELECT FName, LName, Yborn FROM Player WHERE Yborn < (SELECT AVG(Yborn) FROM Player);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT FName, LName, Yborn
                                                 -> FROM Player
                                                 -> WHERE Yborn < (
                                                 ->     SELECT AVG(Yborn) FROM Player
                                                 -> );
+------------+-------------+-------+
| FName      | LName       | Yborn |
+------------+-------------+-------+
| Sachin     | Tendulkar   |  1973 |
| Brain      | Lara        |  1969 |
| Anil       | Kumble      |  1970 |
| Sanath     | Jayasuriya  |  1969 |
| Shane      | Warne       |  1969 |
| Muthiah    | Murlitharan |  1972 |
| Inzamam    | UI-Huq      |  1970 |
| Heath      | Streak      |  1974 |
| Gary       | Kirsten     |  1967 |
| Chaminda   | Vaas        |  1974 |
| Stephen    | Fleming     |  1973 |
| Ricky      | Ponting     |  1974 |
| Shaun      | Pollock     |  1973 |
| Rahul      | Dravid      |  1973 |
| Herschelle | Gibbs       |  1974 |
| Saurav     | Ganguly     |  1972 |
| Adam       | Gilchrist   |  1971 |
| Michael    | Vaughan     |  1974 |
+------------+-------------+-------+
18 rows in set (0.0027 sec)
```

6 [Subquery - ALL Operator]: Write a query to find player batting performances where NRuns is greater than ALL runs scored by players representing 'Australia'.

SELECT b.MatchID, b.PID, b.NRuns FROM Batting b WHERE b.NRuns > ALL (SELECT b2.NRuns FROM Batting b2 INNER JOIN Player p ON b2.PID = p.PlayerID WHERE p.Country = 'Australia');

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT b.MatchID, b.PID, b.NRuns
                                                 -> FROM Batting b
                                                 -> WHERE b.NRuns > ALL (
                                                 ->     SELECT b2.NRuns FROM Batting b2
                                                 ->     INNER JOIN Player p ON b2.PID = p.PlayerID
                                                 ->     WHERE p.Country = 'Australia'
                                                 -> );
+---------+-------+-------+
| MatchID | PID   | NRuns |
+---------+-------+-------+
|    2755 | 25001 |    71 |
|    2755 | 91001 |    60 |
|    2689 | 89001 |    91 |
+---------+-------+-------+
3 rows in set (0.0015 sec)
```

7 [Subquery - ANY / SOME]: Find all bowling performances where Nwickets is greater than ANY wicket count achieved by bowlers from 'Sri Lanka'.

SELECT b.MatchID, b.PID, b.Nwickets FROM Bowling b WHERE b.Nwickets > ANY (SELECT b2.Nwickets FROM Bowling b2 INNER JOIN Player p ON b2.PID = p.PlayerID WHERE p.Country = 'Sri Lanka');

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT b.MatchID, b.PID, b.Nwickets
                                                 -> FROM Bowling b
                                                 -> WHERE b.Nwickets > ANY (
                                                 ->     SELECT b2.Nwickets FROM Bowling b2
                                                 ->     INNER JOIN Player p ON b2.PID = p.PlayerID
                                                 ->     WHERE p.Country = 'Sri Lanka'
                                                 -> );
+---------+-------+----------+
| MatchID | PID   | Nwickets |
+---------+-------+----------+
|    2689 | 99001 |        1 |
|    2689 | 24001 |        1 |
|    2755 | 94002 |        1 |
|    2755 | 92002 |        1 |
|    2755 | 23001 |        2 |
|    2755 | 98002 |        3 |
|    2689 | 98002 |        1 |
+---------+-------+----------+
7 rows in set (0.0019 sec)
```

8 [Correlated Subquery]: Write a correlated subquery to list each player who scored the maximum runs (NRuns) among all players from their own Country.

SELECT DISTINCT p.PlayerID, p.FName, p.LName, p.Country, b.NRuns AS MaxRuns FROM Player p INNER JOIN Batting b ON p.PlayerID = b.PID WHERE b.NRuns = (SELECT MAX(b2.NRuns) FROM Batting b2 INNER JOIN Player p2 ON b2.PID = p2.PlayerID WHERE p2.Country = p.Country);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT DISTINCT p.PlayerID, p.FName, p.LName,
                                                 ->        p.Country, b.NRuns AS MaxRuns
                                                 -> FROM Player p
                                                 -> INNER JOIN Batting b ON p.PlayerID = b.PID
                                                 -> WHERE b.NRuns = (
                                                 ->     SELECT MAX(b2.NRuns)
                                                 ->     FROM Batting b2
                                                 ->     INNER JOIN Player p2 ON b2.PID = p2.PlayerID
                                                 ->     WHERE p2.Country = p.Country
                                                 -> );
+----------+--------+------------+-----------+---------+
| PlayerID | FName  | LName      | Country   | MaxRuns |
+----------+--------+------------+-----------+---------+
|    91001 | Sanath | Jayasuriya | Sri Lanka |      60 |
|    89001 | Sachin | Tendulkar  | India     |      91 |
|    24001 | Andrew | Symonds    | Australia |      42 |
+----------+--------+------------+-----------+---------+
3 rows in set (0.0045 sec)
```

9 [Nested Subquery with HAVING]: Display players whose average runs per match is strictly greater than the overall average runs scored across all matches in the Batting table.

SELECT p.PlayerID, p.FName, p.LName, AVG(b.NRuns) AS AvgRuns FROM Player p INNER JOIN Batting b ON p.PlayerID = b.PID GROUP BY p.PlayerID, p.FName, p.LName HAVING AVG(b.NRuns) > (SELECT AVG(NRuns) FROM Batting);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT p.PlayerID, p.FName, p.LName,
                                                 ->        AVG(b.NRuns) AS AvgRuns
                                                 -> FROM Player p
                                                 -> INNER JOIN Batting b ON p.PlayerID = b.PID
                                                 -> GROUP BY p.PlayerID, p.FName, p.LName
                                                 -> HAVING AVG(b.NRuns) > (
                                                 ->     SELECT AVG(NRuns) FROM Batting
                                                 -> );
+----------+--------+------------+---------+
| PlayerID | FName  | LName      | AvgRuns |
+----------+--------+------------+---------+
|    25001 | MS     | Dhoni      | 53.5000 |
|    91001 | Sanath | Jayasuriya | 60.0000 |
|    89001 | Sachin | Tendulkar  | 91.0000 |
|    24001 | Andrew | Symonds    | 42.0000 |
+----------+--------+------------+---------+
4 rows in set (0.0016 sec)
```

10 [Derived Table in FROM]: Create a derived table in the FROM clause calculating total runs and total balls faced per player, then select players with a strike rate ((Total_Runs / Total_Balls) * 100) greater than 100.

SELECT p.PlayerID, p.FName, p.LName, t.Total_Runs, t.Total_Balls, ROUND(t.Total_Runs * 100.0 / NULLIF(t.Total_Balls, 0), 2) AS StrikeRate

FROM (SELECT PID, SUM(NRuns) AS Total_Runs, SUM(Nballs) AS Total_Balls FROM Batting GROUP BY PID) t

INNER JOIN Player p ON t.PID = p.PlayerID WHERE t.Total_Runs * 100.0 / NULLIF(t.Total_Balls, 0) > 100;

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT p.PlayerID, p.FName, p.LName,
                                                 ->        t.Total_Runs, t.Total_Balls,
                                                 ->        ROUND(t.Total_Runs * 100.0 / NULLIF(t.Total_Balls, 0), 2) AS StrikeRate
                                                 -> FROM (
                                                 ->     SELECT PID, SUM(NRuns) AS Total_Runs,
                                                 ->            SUM(Nballs) AS Total_Balls
                                                 ->     FROM Batting
                                                 ->     GROUP BY PID
                                                 -> ) t
                                                 -> INNER JOIN Player p ON t.PID = p.PlayerID
                                                 -> WHERE t.Total_Runs * 100.0 / NULLIF(t.Total_Balls, 0) > 100;
+----------+--------+------------+------------+-------------+------------+
| PlayerID | FName  | LName      | Total_Runs | Total_Balls | StrikeRate |
+----------+--------+------------+------------+-------------+------------+
|    91001 | Sanath | Jayasuriya |         60 |          52 |     115.38 |
+----------+--------+------------+------------+-------------+------------+
1 row in set (0.0094 sec)
```

11 [Set Operator - UNION]: Write a query using UNION to combine the PlayerIDs of players who scored > 50 runs in Batting with players who took >= 3 wickets in Bowling.

SELECT PID AS PlayerID FROM Batting WHERE NRuns > 50 UNION SELECT PID AS PlayerID FROM Bowling WHERE Nwickets >= 3;

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT PID AS PlayerID
                                                -> FROM Batting
                                                -> WHERE NRuns > 50
                                                -> UNION
                                                -> SELECT PID AS PlayerID
                                                -> FROM Bowling
                                                -> WHERE Nwickets >= 3;
+----------+
| PlayerID |
+----------+
|    25001 |
|    91001 |
|    89001 |
|    98002 |
+----------+
4 rows in set (0.0010 sec)
```

12 [Set Operator / Subquery Intersect]: Find players who both batted AND bowled in MatchID = 2755 using an INTERSECT query or equivalent subquery structure.

SELECT PlayerID, FName, LName FROM Player WHERE PlayerID IN (SELECT PID FROM Batting WHERE MatchID = 2755) AND PlayerID IN (SELECT PID FROM Bowling WHERE MatchID = 2755);

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT PlayerID, FName, LName
                                                -> FROM Player
                                                -> WHERE PlayerID IN (
                                                ->     SELECT PID FROM Batting WHERE MatchID = 2755
                                                -> )
                                                -> AND PlayerID IN (
                                                ->     SELECT PID FROM Bowling WHERE MatchID = 2755
                                                -> );
+----------+-----------+-------------+
| PlayerID | FName     | LName       |
+----------+-----------+-------------+
|    94002 | Chaminda  | Vaas        |
|    92002 | Muthiah   | Murlitharan |
|    91001 | Sanath    | Jayasuriya  |
|    23001 | Yuvraj    | Singh       |
|    98002 | Harbhajan | Singh       |
+----------+-----------+-------------+
5 rows in set (0.0014 sec)
```

13 [Complex Grouping & HAVING]: Display Ground, total runs scored on that ground, and total matches played, considering only grounds where more than 2 matches took place and total runs exceed 100.

SELECT m.Ground, COALESCE(SUM(b.NRuns), 0) AS TotalRuns, COUNT(DISTINCT m.MatchID) AS TotalMatches FROM ODIMatch m LEFT JOIN Batting b ON m.MatchID = b.MatchID GROUP BY m.Ground HAVING COUNT(DISTINCT m.MatchID) > 2 AND COALESCE(SUM(b.NRuns), 0) > 100;

```
MySQL  localhost:33060+ ssl  aarya61_match  SQL > SELECT m.Ground,
                                                 ->        COALESCE(SUM(b.NRuns), 0) AS TotalRuns,
                                                 ->        COUNT(DISTINCT m.MatchID) AS TotalMatches
                                                 -> FROM ODIMatch m
                                                 -> LEFT JOIN Batting b ON m.MatchID = b.MatchID
                                                 -> GROUP BY m.Ground
                                                 -> HAVING COUNT(DISTINCT m.MatchID) > 2
                                                 -> AND COALESCE(SUM(b.NRuns), 0) > 100;
+---------+-----------+--------------+
| Ground  | TotalRuns | TotalMatches |
+---------+-----------+--------------+
| Colombo |       153 |            3 |
+---------+-----------+--------------+
1 row in set (0.0086 sec)
```